

Medium

🔍 Search



Материал о навык... · [Подписаться на публикацию](#)

Искусственный Интеллект +

Программирование +

Разработка программного обеспечения +

Разработка программного обеспечения +

Кодирование +

7 привычек при программировании с использованием ИИ, которые незаметно снижают ваши результаты

11 минут чтения · 1 день назад



Кодовая настройка

Подписаться



Слушай



Поделиться

Опасные привычки — это не очевидные провалы. Это рабочие процессы, которые кажутся продуктивными, но при этом лишают вас понимания, доказательств и ответственности.

7 AI CODING HABITS THAT QUIETLY RUIN YOUR RESULTS



Самые опасные привычки при программировании с использованием ИИ не кажутся опасными.

Они кажутся быстрыми.

Неясный запрос становится полноценной реализацией. Сообщение об ошибке приводит к появлению еще одного патча. Функция появляется в нескольких файлах до того, как разработчик окончательно разберется в требовании.

Код может компилироваться. Тесты могут проходить успешно. Интерфейс может выглядеть работоспособным.

Именно поэтому ущерб легко не заметить.

Слабые рабочие процессы с использованием искусственного интеллекта редко дают сбой в какой-то критический момент. Они незаметно подменяют решения настройками по умолчанию, отладку — регенерацией, проверку — уверенностью, а ответственность — принятием.

Разработчик по-прежнему продуктивен. Он пишет все больше кода. Кажется, что все больше ошибок исчезает. Задачи, на которые раньше уходили часы,

теперь решаются за минуты.

Но результат становится сложнее объяснить, труднее проверить и дороже изменить.

Я больше не оцениваю продуктивность ИИ в программировании по тому, как быстро появляется первое решение. Я оцениваю ее по тому, что остается после остановки генерации.

Может ли разработчик объяснить концепцию? Видны ли допущения? Было ли поведение системы проверено независимо? Может ли другой инженер оценить изменение, не читая переписку, в которой оно было предложено? Может ли кто-то отладить систему, если основной сценарий не сработал?

«Результаты» — это не только код, который выполняется один раз. Хороший результат должен оставаться корректным, поддерживаемым, проверяемым, безопасным, понятным и согласованным с окружающей его системой.

Перечисленные ниже привычки незаметно ухудшают результаты работы ИИ-кодировщиков, поскольку ослабляют защиту, создавая видимость эффективности рабочего процесса.

1. Вы даете запрос до того, как поймете проблему

Многие слабые рабочие процессы с использованием ИИ начинаются с запроса на внедрение.

Разработчик получает требование:

Добавьте отмену подписки.

Они сразу же спрашивают:

Создайте процесс отмены подписки в Node.js.

Название функции звучит конкретно, но проблема с поведением до сих пор не решена.

Можно ли отменить подписку сразу? Сохраняется ли доступ до конца платежного периода? Возможен ли возврат средств? Можно ли отменить подписку? Что делать, если платеж прошел успешно, но обновление локальной базы данных не удалось?

Если таких решений нет, ИИ может заполнить пробелы правдоподобными вариантами поведения.

Сгенерированная реализация может быть внутренне согласованной. В ней могут использоваться понятные названия, логичные модули и привычные шаблоны API. При этом она может кодировать правила, которые бизнес никогда не выбирал.

A stronger workflow begins with behavior:

```
Help me define the behavior for subscription cancellation.  
Known constraints:  
- Access continues until the end of the billing period.  
- No automatic refund is issued.  
- The operation must be idempotent.  
- Existing scheduled renewals must be disabled.  
- The payment provider and local database may become inconsistent.  
Before generating code, identify missing business decisions,  
failure cases, and required state transitions.
```

The important difference is not prompt length.

The second workflow treats ambiguity as an engineering problem instead of allowing the model to resolve it silently.

This matters because an implementation built around the wrong state model becomes expensive to repair. The API contract, database fields, background jobs, provider integration, and user interface may all begin depending on assumptions that were never approved.

By the time the real requirement becomes clear, the generated code is no longer a disposable draft. It has shaped the architecture around incomplete behavior.

AI cannot implement the right behavior when nobody has decided what “right” means.

Before requesting code, define the desired outcome, known constraints, important failure cases, and unresolved decisions. Then use AI to challenge that model before asking it to implement anything.

2. You Dump Uncurated Context Into the Prompt

Developers are often told that better AI answers require more context.

That is generally true. It also creates a dangerous habit:

Paste everything and let the model decide what matters.

A developer may provide ten source files, outdated documentation, old implementations, unrelated logs, competing type definitions, unfinished experiments, and large configuration files.

The model now has more information, but the developer has not necessarily explained which information should govern the decision.

Large context can contain deprecated patterns, contradictory contracts, duplicate implementations, and examples that no longer represent the intended architecture. AI does not always know which source is authoritative.

It may combine a legacy helper with a newer service, copy an obsolete response shape, and use naming conventions from another part of the codebase. The final answer can look coherent while fitting none of those systems completely.

A weak request might say:

Here is the entire authentication folder. Fix refresh tokens.

A stronger workflow provides the relevant controller, token service, session model, exact failure, intended behavior, and constraints that must remain unchanged.

It also identifies authority:

The token service **is** authoritative.
The legacy session helper **is** being removed **and** must **not**

be used **as** an implementation pattern.
Do not change the response contract.
Focus only **on** refresh-token reuse detection.

That final instruction is important because exclusion is part of good context.

Developers often assume that every related file should be included. In reality, irrelevant context creates more opportunities for the model to infer the wrong pattern. Contradictory context is even worse because the model may resolve the contradiction without making the choice visible.

More context is not always better context.

The goal is not to give AI the largest possible view of the system. It is to provide the smallest complete view required for the current decision.

Context becomes useful only after the developer decides what is relevant, authoritative, and intentionally excluded.

Without that curation, more context can produce more confident answers built on less reliable interpretation.

3. You Ask AI to Generate Too Much at Once

Large requests often invite large answers.

A developer asks:

Build authentication for this application.

The model generates registration, login, refresh tokens, password reset, email verification, guards, middleware, database entities, validation objects, configuration, tests, and documentation.

The output looks impressive.

The review burden is enormous.

The developer must now inspect every security assumption, dependency, data shape, error contract, lifecycle rule, framework convention, and failure path introduced across the feature.

The generation may take seconds. Responsible review may take hours.

That gap is dangerous because the amount of code can exceed the developer's capacity to understand it carefully. The workflow then shifts from review to acceptance.

Instead of asking whether every decision fits the system, the developer asks weaker questions:

Does it compile?

Do the tests pass?

Did the model explain it confidently?

A 50-line change can be inspected line by line.

A 2,000-line generated feature encourages trust without comprehension.

The better workflow is incremental. Ask AI to produce the behavior model, affected surface, implementation plan, and assumptions before generating code.

For example:

```
Add refresh-token reuse detection.  
First show:  
1. The required behavior  
2. The smallest affected surface  
3. The files that must change  
4. The assumptions that need verification  
Do not generate the implementation yet.
```

Once the plan is understood, request one small change and its tests. Review it before moving to the next step.

Small changes are easier to compare with existing conventions. They are easier to test, reverse, and debug. They also make architectural disagreements visible before they spread across the codebase.

Oversized generation creates review debt immediately. It may add abstractions nobody requested, dependencies nobody evaluated, and failure behavior nobody noticed.

AI can generate code faster than a developer can responsibly understand it.

Keep each generated change small enough that you can explain every meaningful decision before asking for the next one.

4. You Accept Plausible Code Without Challenging Its Assumptions

AI-generated code often appears complete because its assumptions remain hidden.

Consider this stock-reservation function:

```
async function reserveStock(  
  productId: string,  
  quantity: number,  
) {  
  const product = await productRepository.findOneBy({  
    id: productId,  
  });  
  product.stock -= quantity;  
  await productRepository.save(product);  
  return product;  
}
```

The code is readable. The function name is clear. The implementation is short.

It also assumes the product exists, the quantity is positive, enough stock is available, and the caller is authorized.

It assumes concurrent requests cannot oversell the item. It assumes another update cannot be overwritten. It assumes retries cannot reserve the same stock twice. It assumes stock belongs to only one warehouse and that negative inventory is either impossible or acceptable.

The syntax is finished.

The engineering decision is not.

Get CodeTune's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

The most useful review question is:

What must be true for this code to be correct?

AI can help expose those hidden assumptions:

List every assumption this implementation makes about:

- input validity
- authorization
- concurrency
- database constraints
- retries
- transaction boundaries
- failure recovery

For each assumption, identify whether it should be protected by validation, a type, a database constraint, a transaction, a test, or an explicit business rule.

That request changes the model's role. Instead of only producing code, it becomes a design-review assistant.

Some assumptions belong in validation. Others require database constraints, transactional updates, row locking, idempotency keys, or explicit state transitions. Some may be acceptable as documented limitations.

The point is not to eliminate every assumption. Software always depends on assumptions.

The point is to prevent them from remaining invisible.

Unchallenged plausible code creates race conditions, duplicated side effects, invalid state, inconsistent errors, and security gaps. These failures often appear only under concurrency, retries, partial success, or unexpected input.

Clean syntax can make an unfinished decision look complete.

Before accepting generated code, identify the conditions it depends on and decide which of them must become enforceable system behavior.

5. You Let the Implementation Define Its Own Tests

One of the most subtle AI coding habits is asking a model to generate an implementation and then allowing that implementation to define what correctness means.

A developer asks AI to write a function.

Then they ask:

Write tests for this code.

The generated tests often inherit the same assumptions as the implementation. If the code misunderstood the contract, the tests may formalize that misunderstanding.

Consider pagination:

```
function getOffset(page: number, limit: number) {  
  return page * limit;  
}
```

AI may generate this test:

```
it("calculates the offset", () => {  
  expect(getOffset(2, 20)).toBe(40);  
});
```

The test passes.

But the application uses one-based page numbers. Page one begins at offset zero, which means page two should begin at offset twenty:

```
const offset = (page - 1) * limit;
```

The passing test does not independently verify the behavior. It repeats the implementation's assumption.

A passing test is not automatically independent evidence.

Tests become weak when they mirror the code, test internal details, omit boundaries, use examples selected by the implementation, or ignore the real business contract.

A stronger workflow defines the behavior first:

```
Pagination uses one-based page numbers.  
Required examples:  
- page 1, limit 20 → offset 0  
- page 2, limit 20 → offset 20  
- page 5, limit 10 → offset 40  
- page less than 1 must be rejected
```

The implementation can then be generated against that contract. The tests protect behavior that existed before the code, rather than explaining whatever code happened to be produced.

AI can still generate the tests, but the expected behavior should be defined independently before the implementation exists.

Verification should also come from outside the generated conversation. Existing integration tests, official documentation, type checking, database constraints, boundary testing, code review, and observed behavior all provide different forms of evidence.

When the same assumption writes both the code and the test, passing tests may prove consistency rather than correctness.

Define the contract first. Let tests defend that contract instead of validating the model's own interpretation of it.

6. You Prompt for Another Patch Instead of Debugging

The prompt-and-retry loop feels productive.

The code fails.

The developer pastes the error into AI.

The model changes several lines.

A different error appears, so that error is pasted back. Another patch is generated. After several rounds, the feature seems to work.

The final result may run, but the developer often cannot explain what caused the original failure, which patch fixed it, or which changes were unnecessary.

They may not know whether the solution is stable, whether a hidden symptom remains, or what will happen when the environment changes.

The code is repaired, but understanding has been traded away.

A weak debugging request says:

Fix this error.

That request encourages another solution before the problem has been understood.

A stronger debugging request preserves the investigation:

Expected behavior:
[describe it]

Actual behavior:
[describe it]

Evidence:
[stack trace, logs, state, recent changes]
Do not rewrite the code.

Give me three possible root causes ranked by likelihood.
For each hypothesis, suggest one observation or experiment
that would confirm or reject it.

Now AI assists debugging rather than replacing it.

The developer still follows the real process:



AI's ranked causes are not evidence. They are suggestions about where evidence might be collected.

That distinction matters because a model can produce a plausible explanation without knowing which condition actually occurred. The next step should be an observation, breakpoint, focused log, test, or controlled experiment that rejects or supports the hypothesis.

Prompt-and-retry debugging creates patch accumulation. Unnecessary changes remain because nobody knows which one mattered. Logic is duplicated. Regressions appear later. The developer becomes increasingly dependent on the model because each unexplained patch makes the system harder to reason about.

The feature may start working while the developer becomes less able to explain why.

When code fails, stop requesting solutions long enough to define the expected behavior, actual behavior, first point of divergence, a current hypothesis, and the smallest experiment that could confirm or reject it.

7. You Keep Code You Cannot Explain or Defend

AI will not approve the pull request.

It will not respond to the production incident, explain a security decision, defend the architecture during review, or maintain the feature six months later.

The developer who accepts the code owns those responsibilities.

A useful standard is:

Do not merge AI-generated code you could not defend if a reviewer questioned every important decision.

That means being able to explain why the code exists, why the design was chosen, and which alternatives were rejected. It means understanding the assumptions, failure paths, retry behavior, verification strategy, and likely effect of future requirements.

There are clear signs that ownership has been lost.

You cannot explain an abstraction the model introduced. You avoid renaming anything because you are afraid of breaking it. You depend on the original conversation to understand the implementation. You do not know why a dependency was added or how the code handles invalid input.

You would need AI to explain the change during code review.

At that point, the work is not finished.

Code without ownership becomes difficult to maintain, debug, and modify because nobody knows which decisions or behaviors were intentional.

The original AI conversation can also become an undocumented dependency. The repository contains the output, but not the reasoning, rejected alternatives, or context that produced it.

Generated code should be treated as raw material, not authority.

Delete abstractions that do not earn their place. Rename misleading concepts. Remove dependencies that solve problems the system does not have. Rewrite code that works but does not fit the architecture.

The responsibility does not move to the model just because the first draft came from one.

Before accepting generated code, ask:

Could I explain this change without reopening the AI conversation?

If the answer is no, the implementation may exist, but the engineering work is incomplete.

The Results Were Ruined Before the Code Broke

The most dangerous AI coding habits do not always produce broken code.

They produce code that works just well enough to avoid immediate suspicion.

The requirement was never fully defined, but the implementation looked complete. The context was contradictory, but the answer sounded confident. The change was too large to review properly, but the tests passed. Debugging disappeared into a sequence of patches, but the final error was gone.

Each step felt productive.

Together, they weakened the result.

Bad AI coding habits do not always ruin the first version. They ruin your ability to trust, change, review, and recover from that version.

Good AI-assisted coding is not measured by how much code appears or how quickly the first solution runs. It is measured by what the developer still understands after generation ends.

Define the behavior. Curate the context. Keep the change small. Expose assumptions. Verify from an independent contract. Debug through evidence. Own every line that enters the system.

AI can remove repetitive work from software development. It can accelerate exploration, implementation, documentation, and review.

That is valuable.

It should not remove the reasoning that makes the work trustworthy.

The habit that quietly ruins AI coding results is always some version of the same mistake:

allowing faster output to replace engineering control.

- Artificial Intelligence
- Programming
- Software Development
- Software Engineering
- Coding



Follow

Published in Skill Stuff

770 followers · Last published just now

Skill Stuff is your go-to hub for coding tips, problem-solving strategies, and developer growth insights.



Follow

Written by CodeTune

282 followers · 2.2K following

Full-stack engineer turning complex ideas into simple, high-performance products.
